醫學影像分析專題

HW1

作者：

資工二 412210037 林威岑

113 年 4 月 3 日

目錄

一、醫學背景 3

1. 介紹此 Kaggle 競賽： 3

2. 解釋相關醫學名詞： 6

二、Data 介紹 8

1. 介紹 HW 的資料 8

2. 隨機顯示 2 張 json 檔案標註在 TIF 圖片的樣子 8

三、演算法與程式碼介紹 9

1. 程式碼架構流程： 9

2. 程式碼介紹： 10

1. Upload Training Data 上傳訓練集資料 10

2. Label The Training Data by Json File 將訓練集資料進行標記 11

3. Data Augmentation 數據增強 13

4. Preprocess The Training & Testing Data 前處理：影像顏色標準化 14

5. Split The Data 分割數據集：85%訓練集 / 15%驗證集 16

6. Setup U-Net Structure 建構 U-Net 神經網路 17

7. Start Training with U-Net 開始訓練模型 19

8. Predict the testing data 使用模型預測測試資料 20

9. Transfer Predict Results to Label Data 將預測結果轉換成標記資料 21

3. 介紹演算法－U-Net： 22

四、成果 23

五、討論與感想 24

六、參考資料 25

此作業摘自 Kaggle 競賽－Segment instances of microvascular structures from healthy human kidney tissue slides。本作業的目標是在 MATLAB 上由 test 測試集 10 張照片生成出各照片中 blood_vessel(血管)的位置，將這些血管位置以 json 檔格式儲存。資料下載：Data download(872MB)

一、醫學背景

1. 介紹此 Kaggle 競賽：

- **題目：**破解人類血管系統
- **競賽目標與說明：**
  自動分割人體腎臟組織的微血管結構，包含毛細血管、小動脈以及小靜脈。將使用 2D PAS 染色的腎臟組織切片進行訓練並建立出模型。
- **背景：**
  此次目標是此競賽的主辦單位－人體生物分子圖譜計畫（HuBMAP）的一部分，此計畫致力於開發一個開放的繪製健康人體細胞圖譜的全球平台，利用最新的技術研究全身細胞之間的關係。而目前繪製細胞圖譜的工作涉及脈管系統通用座標框架（VCCF, Vasculature Common Coordinate Framework），這個框架使用了人體內的血管系統作為主要導航系統，提供了一種以毛細管結構為位址來識別細胞位置的獨特方法。

  然而，現在研究人員對於脈管結構還不甚了解，所以希望透過此次競賽自動分割出微血管結構，幫助研究人員對血管在人體組織中如何排列更加理解，最終目標是促進脈管系統通用座標框架（VCCF）和人體參考圖譜（HRA）的建立，理解細胞之間的關係如何影響我們的健康。
- **數據集：**
  - **完整數據架構：**
    此競賽中提供的數據資料來自完整的五張 WSI（Whole Slide Image）全視野切片影像，而這些影像主要被分成兩個數據集－經過專家審查並標註資料的是 Dataset 1；標註較少且未經過專家確認的則是 Dataset 2。還有一組 Dataset 3，影像內容來自另外 9 張 WSI，並且裡面沒有任何

標記資料。

- **數據分配與安排：**
  - **訓練集：**來自 2 張 WSI，包含 Dataset 1,2 的資料（不包含私有測試集的資料）
  - **測試集：**分成公開以及私有。其中公開測試集的資料來自 2 張 WSI；私有測試集的資料來自 1 張隱藏的 WSI。（參賽者只能獲取部分的測試數據，完整的測試集不會分開）
- **數據內容：**
  - Train, Test 的資料夾中分別存放訓練集以及測試集的 TIFF 影像。影像的資料為 512 * 512 * 3 的像素切片 。
  - Polygons 的 json 檔案中存放多邊形的分割標註資料：
    - **Id**：影像的識別碼
    - **Annotations**：遮罩資訊，其中包含
      - **type** 標記的類型：blood_vessel 血管、glomerulus 腎小球、unsure 不確定
      - **coordinate**：標記分布在影像中的座標
  - 三份 csv 檔案，其中 tile_meta 中存放訓練集資料對應的 dataset、WSI 資料來源以及完整 WSI 中的位置；另一份 wsi_meta 檔案中則是存放 WSI 的來源的個人資訊；以及 sample_submission 存放正確格式的範例繳交檔案。

- **相關生物背景知識：**
  - **腎臟區：**

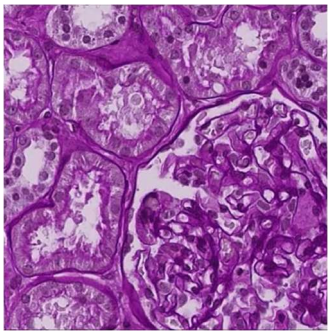
    腎臟在人體的主要功能是調節與過濾血液，透過尿液將廢物與多餘的體液排出體外。

    由於此次競賽的資料集來自人類腎臟的 Periodic acid-Schiff（PAS）染色組織切片。其中組織切片是 3D 結構的 2D 切面，因此切片可能會顯示腎單位和血管結構的水平或縱向切面。而 PAS 染色會標記富含碳水化合物的結構，例如腎單位內的毛細血管的基底膜和腎小管上皮細胞，在此資料集中也會有以下組織樣本

    - **腎皮質（Renal cortex）**

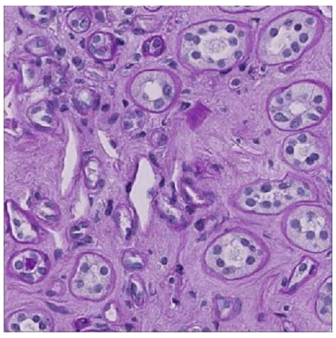
      腎皮質是腎臟的外部區域，包含圓形的腎小囊（renal

corpuscles），其中包裹著腎小球（glomerular tuft），即毛細血管環狀結構。腎小囊是腎單位的起點，血液在此開始過濾。腎皮質還包含近曲小管（PCT, proximal convoluted tubules）和遠曲小管（DCT, distal convoluted tubules），這些也是腎單位的一部分。在這些小管結構之間，存在一個複雜的毛細血管網絡，稱為腎小管周圍毛細血管（peritubular capillaries）。

 <- 腎皮質的圖（包含部分腎小球結構）

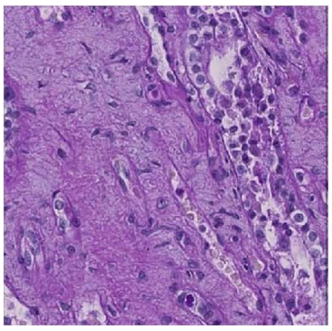
- **腎髓質（Renal medulla）**

  腎髓質是腎臟的內部區域，由 8 至 15 個腎錐體（renal pyramids）構成，其內含有呈線性排列的小管，如亨利環（loops of Henle）和收集管（collecting ducts），這些管道負責收集待排出的尿液。腎髓質內的毛細血管網絡主要由直小管（vasa recta）構成。腎錐體之間由腎皮質的延伸部分——腎柱（renal columns）分隔。此外，腎髓質還有向外突入腎皮質的結構，稱為髓放射（medullary rays）。

 <- 腎髓質的圖

- **腎乳頭（Renal papilla）**

  髓質的一部分。腎錐體的寬闊底部與腎皮質在皮質髓質交界處（corticomedullary junctions）相連，而腎錐體的頂端則形成腎乳頭（renal papilla）。這些腎乳頭向小腎盞（minor renal calyces）延伸，並將尿液收集至腎盂。

 <- 腎乳頭的圖

- **血管區：**

  攜帶氧氣的動脈進入器官或組織後，會逐漸分支成越來越小的結構，最終形成小動脈（arterioles）。小動脈進一步分支形成毛細血管網絡。毛細血管（capillaries）的直徑約為 8-10 微米，呈管狀結構，由一層薄薄的內皮細胞構成，這層內皮足夠薄，使紅血球中的氧氣和營養物質能夠穿透並進入周圍組織。毛細血管最終匯聚成小靜脈（venules），將缺氧和營養不足的血液運出該區域。這些小靜脈進一步合併成更大的靜脈（veins）。

  此競賽專注於腎臟的微血管系統，包括小動脈、毛細血管和小靜脈。較大的血管結構（如動脈和靜脈）未在此數據集中標註。

- **資料來源：**HuBMAP - Hacking the Human Vasculature

2. 解釋相關醫學名詞：

1. **microvascular structures：**

   微血管結構指的是人體內最小的血管，包括毛細血管、小動脈和小靜脈。這些血管負責血液與組織之間的氣體交換、營養物質運輸及廢物排除，在組織微循環中扮演關鍵角色。

2. **capillaries：**

   毛細血管。是人體內最細小的血管，僅由一層內皮細胞構成。毛細血管壁非常薄，使氧氣、二氧化碳、營養物質和廢物能夠在血液和組織細胞間自由交換。

3. **arterioles：**

   小動脈。是由較大的動脈（arteries）分支而成，直徑較小，負責將富含氧氣的血液輸送至毛細血管網絡。小動脈的平滑肌層較厚，能夠調節血管直徑，從而控制血流量和血壓。

4. **venules：**

小靜脈。是毛細血管網絡的出口，負責收集缺氧血液並輸送至較大的靜脈（veins），最終回流至心臟。它們的結構比小動脈更薄，主要作用是讓血液順利回流而不發生逆流。

5. **2D PAS-stained histology images：**

   2D PAS 染色的組織切片影像。這些影像是 2D 平面的病理組織學圖像。PAS 染色（Periodic acid-Schiff stain）是一種常見的組織染色技術，能夠染色富含醣類的細胞結構。

6. **blood vessels：**

   血管是人體內運輸血液的管道。主要分為動脈、靜脈和毛細血管。血管的主要功能是將氧氣和養分輸送至各個組織，並將代謝廢物帶回至排泄系統。

7. **VCFF（Vasculature Common Coordinate Framework）：**

   一個標準化的空間座標系統。用於統一標記與分析人體內的血管結構。在醫學影像分析中，這種框架有助於不同數據集之間的比較，並幫助研究人員識別與定位特定的血管結構。

8. **HuBMAP（Human BioMolecular Atlas Program）：**

   人類生物分子圖譜計畫。是美國國家衛生研究院主導的一個計畫，目標是建立一個高解析度的細胞和組織圖譜，幫助研究人員理解人體內各種細胞類型的空間分布和分子特徵，並應用於醫學研究與精準醫療。

9. **HRA（Human Reference Atlas）：**

   一個標準化的人體組織與細胞圖譜。目的是提供一個通用的解剖學參考，幫助研究人員在不同數據集和研究中進行比對與分析。

10. **Whole Slide Image（WSI）：**

    全視野切片影像。是透過數位病理掃描儀拍攝的高解析度組織切片影像，能夠涵蓋整個顯微鏡視野，並允許進行數位化分析和自動病理診斷。隨著科技的發展與進步，此數位化影像可基於人工智慧演算法實現初步的病理診斷。

11. **TIFF：**

    TIFF 是一種高品質的影像存儲格式。常用於醫學影像和科學計算中。支援無損壓縮，適合存儲高解析度的病理切片圖像。

12. **Polygonal Segmentation：**

    多邊形分割。一種影像分割技術，用於標記影像中的特定區域，特別是在醫學影像分析中，研究人員會使用多邊形標註微血管、腎小球等結構，以便後續進行機器學習訓練或統計分析。

13. **Glomerulus：**

腎小球。是腎臟腎單位中進行血液過濾的關鍵結構，由一團毛細血管環組成，負責過濾血液中的水分、離子和小分子物質，形成原尿。

二、Data 介紹

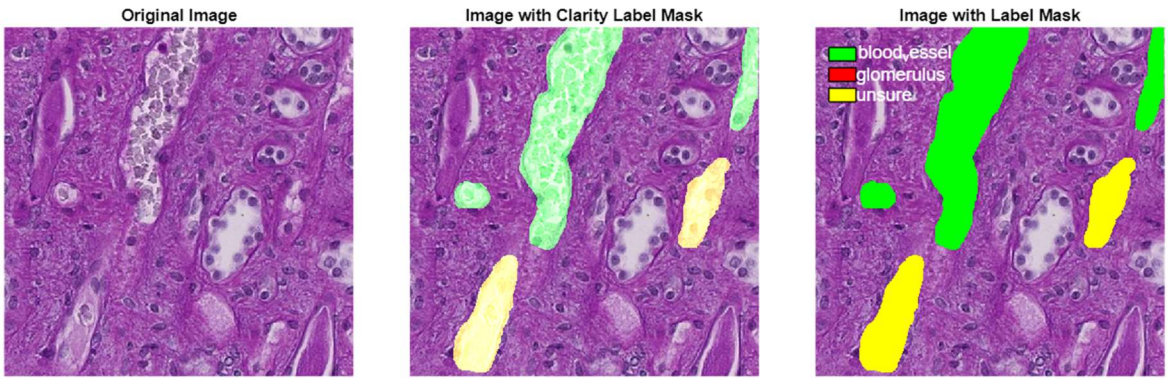
1. 介紹 HW 的資料

此次作業中使用的數據集有

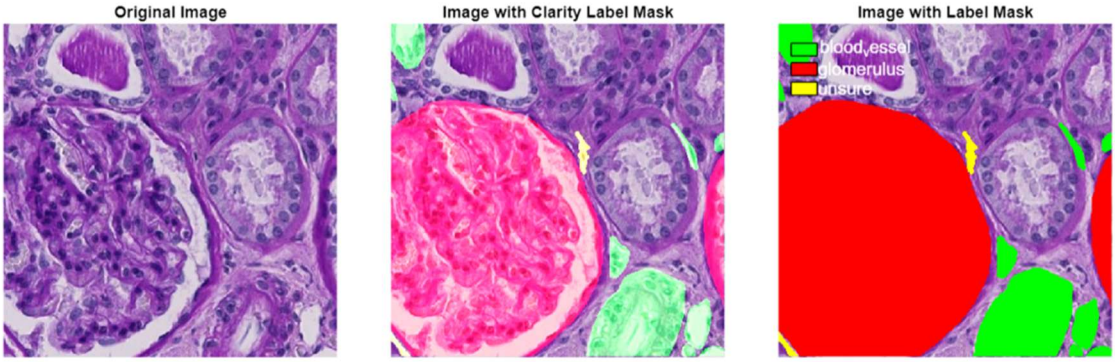
- **（train）訓練集**：1500 張 512 * 512 * 3 的 TIFF 檔案影像
- **（json）資料集**：訓練集中 1500 張照片的對應標註資訊
- **（test）測試集**：10 張 512 * 512 * 3 的 TIFF 檔案影像
- **tile_meta.csv**：存放訓練集資料對應的 dataset、WSI 資料來源以及完整 WSI 中的位置
- **wsi_meta.csv**：存放 WSI 來源的個人資訊，像是性別、年齡、BMI 等

2. 隨機顯示 2 張 json 檔案標註在 TIF 圖片的樣子

第 25 張照片：



第 148 張照片：



三、演算法與程式碼介紹

1. 程式碼架構流程：

1. Upload Training Data 上傳訓練集資料
2. Label The Training Data by Json File 將訓練集資料進行標記
3. Data Augmentation 數據增強
4. Preprocess The Training & Testing Data 前處理：影像顏色標準化（沒做到
5. Split The Data 分割數據集：85%訓練集 / 15%驗證集
6. Setup U-Net Structure 建構 U-Net 神經網路
7. Start Training with U-Net 開始訓練模型
8. Predict the testing data 使用模型預測測試資料
9. Transfer Predict Results to Label Data 將預測結果轉換成標記資料

2. 程式碼介紹：

1. Upload Training Data 上傳訓練集資料

我預設是將此程式碼檔案與此作業的資料放在一起執行，所以透過抓取此檔案當前的位置，進而去拿到 train 訓練集和 json 資料集的路徑，接著我們就可以透過路徑讀取到資料。

1. Upload Training Data

我預設是將這份程式碼放在老師提供的資料夾下，並且已經解壓縮過，所以會直接從這份檔案的位置抓到其他test,json,train,csv等資料。

匯入訓練集資料，包含1500張訓練集的TIF照片，以及對應的1500份json檔案。而其中因為只有來自第一個資料集所標註的資料可信度大，所以我並不打算採用WSI的第二個資料集做使用。

```
% 找到目前檔案的位置
currentFile = matlab.desktop.editor.getActiveFilename;
currentFolder = fileparts(currentFile);

% 設定train,json檔案的路徑
trainFolder = fullfile(currentFolder,"train");
jsonFolder = fullfile(currentFolder, "json");
train_img = dir(fullfile(trainFolder, '*.tif'));

% 預先分配儲存資料的空間
train_num = length(train_img);
imageData = cell(1, train_num);
jsonData = cell(1, train_num);
data_num = 1;

% 先匯入資料集中的資料，方便我們在讀取的時候直接篩選
metaData = readtable("tile_meta.csv");

% 讀取所有圖片,json並儲存到 cell array
for i = 1:train_num
    imagePath = fullfile(trainFolder, train_img(i).name); % 取得照片的路徑
    id = extractBefore(train_img(i).name, '.tif');
    col_idx = find(strcmp(metaData{:,1}, id));
    if(metaData{col_idx,3}==1) % 只讀取第一個資料集的資料
        % 讀取圖片
        imageData{data_num} = imread(imagePath);

        % 讀取 json檔案
        jsonName = strcat(id, '.json'); % 將照片檔名轉成json檔名
        jsonPath = fullfile(jsonFolder, jsonName); % 取的json的路徑
        jsonText = fileread(jsonPath);
        jsonData{data_num} = jsondecode(jsonText); % 解析 JSON
        data_num = data_num+1;
    end
end

data_num = data_num-1;
fprintf("upload all tif img\n");
```

```
upload all tif img
```

而由於前面已經說明過提供的資料中會有兩種可能的 dataset，其中 dataset 1 的資料是經過專家驗證的，而 dataset 2 則沒有經過專家驗證。所以我決定只讀取 dataset 1 的資料作為我的訓練資料集。我先透過讀取 train 資料夾內的影像，取得影像名稱也就是 id 後，再根據 tile_meta 的資料確認此張是否為第一資料集，才讀入影像與對應的 json 檔案資訊。在我們作業的資料集中，總共只有 289 張影像是來自第一個資料集。

2. Label The Training Data by Json File 將訓練集資料進行標記

接下來要開始上標記，我們先將可能的 type 類別定義好名稱以及遮罩的顏色，並設定遮罩的透明度。也要記得要宣告等等要放資料的空間，在這邊我創建了 3 個新的資料，分別存放不同的遮罩資料，annotationMasks 中存放的是每一個 pixel 對應的 type，也就是 0~3 的數值；而 annotatedImage 中我存放了兩種遮罩型態的資料，第一種是在照片中直接覆蓋顏色上去有標記的區域，而第二種不同於第一種是遮罩有透明度，可以依稀看的到原本圖片的輪廓。

2. Label The Training Data by Json File

```
% 定義標籤類別與固定顏色 (RGB 格式，範圍 0~1)
class_type = ["blood_vessel", "glomerulus", "unsure"];
class_colors = [0 255 0;   % blood_vessel (綠色：想要的區域)
                255 0 0;   % glomerulus (紅色：不想要的區域)
                255 255 0];  % unsure (亮黃色：不重要)

 % 設定遮罩的透明度
alpha = 0.7;

% 初始化 Mask, GroundTruth 照片
annotationMasks = cell(1, data_num);  % 建立一個長度為 290 的 cell 陣列
annotatedImage = cell(2, data_num); % (1,:)是直接覆蓋區域的圖；(2,:)的遮罩會呈現半透明

for i = 1:data_num
    annotationMasks{i} = zeros(512, 512, 'uint8');  % 每個 cell 存一個 512×512 的 mask
    annotatedImage{1,i} = imageData{i}; % 將 annotatedImage 初始化為原始圖像
    annotatedImage{2,i} = imageData{i};
end


for i=1:data_num
    % 讀取json檔案
    data = jsonData{i};

    % 解析 JSON 標註
    annotation_num = length(data.annotations);
    for j = 1:annotation_num
        annotation = data.annotations(j);
        labelType = annotation.type; % 例如 'glomerulus'
        classIndex = find(ismember(class_type, labelType));

        % 提取標註的座標
        coordinates = annotation.coordinates; % 取得像素點
        Points_num = length(coordinates);
        x = coordinates(:,:,1); % x 座標
        y = coordinates(:,:,2); % y 座標

        annotationMasks{i} = annotationMasks{i} + uint8(poly2mask(x,y,512,512))*classIndex;
    end

    % 將 RGB 顏色套用到對應的標籤
    for j = 1:3
        % 找出對應標籤的位置
        mask = (annotationMasks{i} == j);

        % 將對應位置的 RGB 值改為指定顏色
        annotatedImage{1,i}(repmat(mask, [1 1 3])) = repmat(class_colors(j,:), [sum(mask(:)), 1]);
        for c = 1:3
                color = double(class_colors(j, c));  % 轉換顏色為 double
                annotatedImage{2,i}(:,:,c) = uint8( double(annotatedImage{2,i}(:,:,c)) + alpha * color * double(mask) );
        end
    end
end
```

接著就是根據 json 檔案的資料標記區域，因為檔案中提供的是標記區域的外圍座標，

所以在這裡我是使用 poly2mask 這個 matlab 內建的函數，只要提供 x,y 的點座標就可以將點自動連成多邊形，並形成一個遮罩，非常的方便。

接著就是根據對應的 type 資料放上對應的顏色這樣就好了！

在程式碼中的下一個 Section 中我也有寫一個可以方便我們看到對應圖片的遮罩圖片，如果有想要看某一張照片的標記資料，只要在 index 的地方輸入想要看的照片索引值就好了，但是要注意範圍只有 1~289，因為我前面只篩選第一個資料集的影像作為我們的訓練資料。

以下這一區段程式碼不一定要執行，只是可以讓我們清楚看到標記後的成果，也就是我們的 **Ground Truth** 資料。
透過賦值 1~289 的數字給 index，就可以觀察到對應 data 的 Ground Truth 資料。

```
% 顯示某index的圖片
index = 148;
figure;
set(gcf, 'Position', [100, 100, 1200, 400]);  % 增大圖形視窗大小

% 設置圖片顯示區域的大小
subplot('Position', [0.05, 0.1, 0.28, 0.8]);  % 左邊圖形
imshow(imageData{index});
title('Original Image');

subplot('Position', [0.36, 0.1, 0.28, 0.8]);  % 中間圖形
imshow(annotatedImage{2,index});
title('Image with Clarity Label Mask');

subplot('Position', [0.67, 0.1, 0.28, 0.8]);  % 右邊圖形
imshow(annotatedImage{1,index});
title('Image with Label Mask');

% 手動添加 Legend (顯示在圖片外部)
% 設定文字和顏色方塊的起始位置
legend_x = 20;  % x 座標 (將標註置於圖片外側)
legend_y = 30;  % 初始 y 座標
box_height = 20;
box_width = 40;
gap = 10; % 標籤間的間距

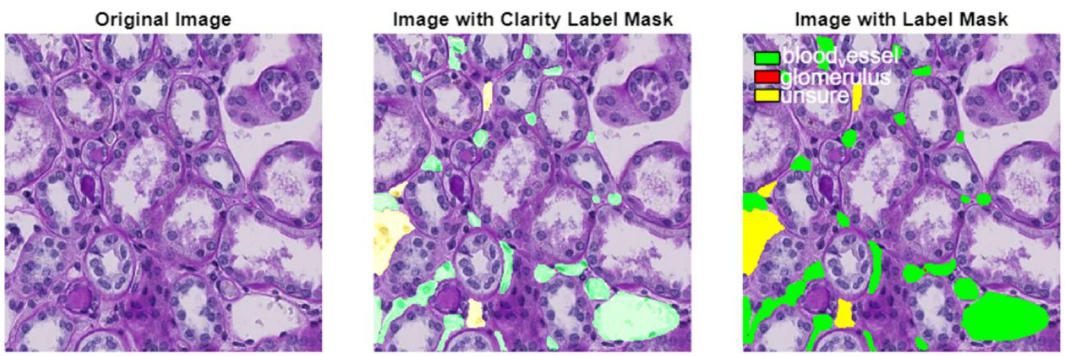
for j = 1:3
    % 畫出顏色方塊
    rectangle('Position', [legend_x, legend_y, box_width, box_height], ...
              'FaceColor', class_colors(j,:) / 255, 'EdgeColor', 'k');

    % 加上標籤名稱
    text(legend_x + box_width + 0.5, legend_y + box_height / 2, class_type(j), ...
         'Color', 'w', 'FontSize', 12, 'VerticalAlignment', 'middle');

    % 調整 y 座標，讓下一個標籤往下排列
    legend_y = legend_y + box_height + gap;
end

hold off;
```

成果圖如下：



3. Data Augmentation 數據增強

這是一個非常重要的步驟！由於我們現在的訓練集資料只有 289 份，這對於要訓練一個辨識微血管區域的模型是十分不夠的，所以我們會透過旋轉、平移、伸縮以及鏡像來擴增我們的訓練集資料。在這邊我將每一張原始的照片都再經過隨機調整生成另外 5 張照片加進我們的資料集中。

3. Data Augmentation

數據增強，這是為了拓展我們的訓練資料集，由於現在的訓練資料只有289份，完全不夠我們進行訓練。

所以我們透過旋轉、平移、鏡像新增更多照片到我們的訓練集中。當然label mask的資料也要同步進行增加。

```
all_data_num = data_num * 6;
trainData = cell(1, all_data_num);
groundTruthData = cell(1, all_data_num);

for i=1:data_num
    trainData{1+(i-1)*6} = imageData{i};
    groundTruthData{1+(i-1)*6} = annotationMasks{i};
end

augmenter_img = imageDataAugmenter('FillValue',0,'RandRotation',[-15,15],'RandScale',[0.8,1.2], 'RandXReflection',true, 'RandYReflection',true);
augmenter_mask = imageDataAugmenter('FillValue',0,'RandRotation',[-15,15],'RandScale',[0.8,1.2], 'RandXReflection',true, 'RandYReflection',true);
% 因為 mask 中本來就有標記的class number
% 分別是1:blood_vessel, 2:glomerulus ,3:unsure，我們將旋轉後產生的黑邊填上第4類，方便我們之後做篩選

for i=1:data_num
    for j=1:5
        augmented_img = augmentedImageDatastore([512,512,3],imageData{i},'DataAugmentation',augmenter_img);
        augmented_mask = augmentedImageDatastore([512,512,3],annotationMasks{i},'DataAugmentation',augmenter_mask);
        trainData{1+(i-1)*6+j} = imtile(preview(augmented_img).input);
        groundTruthData{1+(i-1)*6+j} = imtile(preview(augmented_mask).input);
    end
end
```

其中，經過旋轉的地方，我原本打算在遮罩中新增一個第 4 類去紀錄旋轉後所產生的黑邊，也就是將旋轉後的空白區域填上'4'，不然我怕會影像到後續的模型訓練。不過後來在訓練好幾次模型之後發現，這個黑邊的資料對於我現在設計的模型架構似乎沒有意義，所以最後我又修改成將旋轉後的遮罩資料填充為 0，就當成一般背景忽略就好了。也就是現在看到的程式碼。

反思：若是將旋轉的角度以及放大的倍率調得更寬更大，這樣對模型的訓練會有幫助嗎？

因為若是太多相同或是太少的照片就容易使的模型過度擬合，也就是太貼近 training data 的 case，而導致遇到新的圖片會辨別不出來，新增一些看起來跟原本不太一樣的照片可以幫助訓練，這就是為甚麼我們要做這一個 Part。不過似乎改變太多也會造成一些問題，考慮到這個 2D PAS 的染色病理影像切片資料，切出來微血管的方向可能會是固定的，所以旋轉太多可能會產生不合理的影像，尤其是微血管可能會有特定的組織排列，而放大太多可能也會讓我們損失一些重要的資訊。

4. Preprocess The Training & Testing Data 前處理：影像顏色標準化

觀察一些數據中的影像後，會發現有些染色圖的顏色比較深、有些比較淺，這樣對於模型的學習也會造成問題，因為若是模型把某種顏色當成判斷特徵，那麼當測試的影像顏色跟訓練的模型不太一樣的話，模型就有很大的機率會產生很差的預測結果。

所以針對這個染色影像顏色不統一的問題，有以下兩種解決辦法，一種是將數據增強，不同於上一步，數據增強除了可以透過圖片的旋轉、平移、伸縮等等，我們也可以丟入更多顏色變化的圖片，這樣做可以讓模型學習到微血管的組織結構長怎樣，而非透過顏色作為判斷去辨識；而另一個方法就是將所有影像的顏色分布統一，也就是我現在想要做的－影像顏色標準化。

4. Preprocess The Training & Testing Data（我不確定有沒有經過這個處理，判別準確率會不會上升）

接著要將圖片的色彩標準化，在這邊我們使用Vahadane Stain Normalization，因為MATLAB中沒有這個內建的函數，並且這個函數需要跑較久的時間，我有上網查到一個可以使用GPU加速的Vahadane的開源程式碼，我已自行將它下載下來，並且附在這份程式碼的資料夾中，所以可以直接使用。

我打算以測試集資料的第一張照片作為顏色統一的標準...

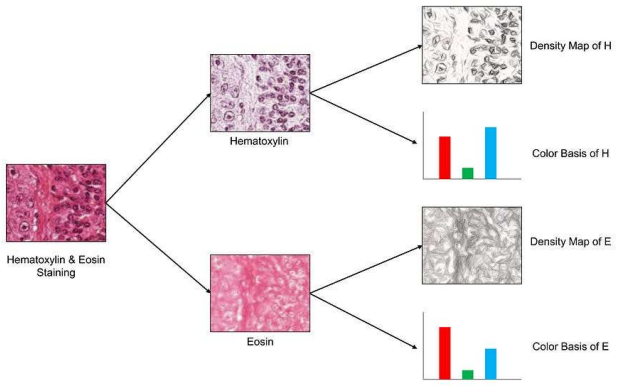
```
% 匯入測試集資料
test_imds = imageDatastore('test',"FileExtensions",'.tif');

% 將當前目錄加入python搜尋路徑
% if count(py.sys.path,'') == 0
%     insert(py.sys.path,int32(0),'');
% end

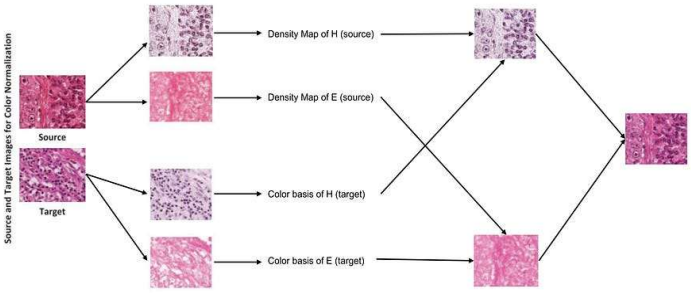
% 需在自己的python環境中下載spams才能正確執行；其實作者也有說可以忽略spams造成的錯誤，不然下載spams很麻煩，會出現一堆錯誤

% 先設定好要輸入的參數（先嘗試做一組）
% osh = trainData{1};
% normalizer = TorchVahadaneNormalizer()
% mask = 1-(groundTruthData{1}==4) % 將旋轉後黑色的部分排除
```

其中，針對染色病理影像的顏色標準化的方法有好幾種，常見的有 Reinhard、Macenko、Vahadane 這三種 stain normalization，而經過查找一些資料後，發現 Vahadane 似乎更適合用來處理病理照片[1]，所以我原先打算使用這個方法來將我的影像進行顏色標準化。



Vahadane 的運作原理大致上是基於一張影像的染色、汙點（stain）結構以及顏色的分布去做處理。而其中我們是透過 H&E stain separation，也就是透過計算非負矩陣分解（NMF）可以得到分離後的資料。[2]



而標準化過程就是將原影像（Source Image）、目標影像（Target Image）各自拆分成上述所說的資訊，接著將原影像的 H,E 染色結構分別和目標影像的 H,E 顏色分布做結合，保留了原始影像的染色結構，並依據目標影像的顏色分布，得到標準化後的影像。

然而在嘗試了幾次之後，我最終選擇放棄使用這個方法，有以下幾個原因：

首先，MATLAB 中沒有 Vahadane 的相關內建函數，所以必須要透過外部的資源才可以做到，而我有上網查到了一個開源的 Vahadane 函數並且支援 GPU 加速[4]，透過 git 將此開源程式碼下載下來後，才發現裡面有一個函數需要使用到 spam 這項工具，而下載 spam 的過程中出現非常多的錯誤，嘗試無數次都沒有成功之外，在考慮到時間的因素後，所以便選擇先暫時省略這一步驟。

另一個原因則是，在大致了解此染色標準化的做法之後，我也才發現這個處理方法是基於 H&E 的染色結構才有的染色分離技術，然而我們的影像是 PAS（Periodic acid-Schiff stain）的染色影像，我沒有辦法確保此 H&E stain separation 對於 PAS 染色也會有相同的效果。因此基於上述種種原因，我最後放棄使用這個方法。

```
% 將mask中classtype >1 的都歸類成0
for i=1:data_num
    groundTruthData{i} = (groundTruthData{i}==1);
end
```

而最後這一部分，我除了匯入測試集資料之外（因為我原本也預計要對測試集資料作顏色標準化），我也將所有標記的資料除了血管以外全部都歸類成背景，這是因為當我後面在訓練幾次模型之後，我發現其他的標記資料似乎對我們沒有什麼幫助，所以乾脆就把其他標記資料（glomerulus、unsure）也都歸類成背景，這樣模型的訓練就會變成只是一個二分類的問題。

5. Split The Data 分割數據集：85%訓練集 / 15%驗證集

當我們都處理完所有數據之後，就可以將我們的數據劃分成訓練集以及驗證集，其中驗證集的存在是為了在每一次訓練模型完後幫助我們評估模型的效能進而去調整參數 [5]。而由於我們的實際資料集很小，大部分都是相似的，所以有別於一般將資料劃分成 8:2 的比例，我選擇分成 85%作為訓練集、15%作為驗證集，這樣做可以讓模型在訓練時有更多的機會學習到不同的架構，而不要把資料浪費在作為驗證集。

5. Split The Data

我們將目前的訓練資料再分成 85%訓練集 以及 15%驗證集，透過隨機抽取。（其實觀察前幾名的做法中，有一位有說到其實他把距離較近的資料分開成訓練集以及驗證集（可能因為相近的架構比較像吧），這樣做才可以讓模型學習到更多不同的照片資訊，不過我沒有這樣做

```
% 設定隨機種子，確保每次劃分結果一致
rng(0);

% 產生隨機索引
indices = randperm(all_data_num);

% 設定 85% 訓練集，15% 驗證集
trainCount = round(0.85 * all_data_num);

% 劃分訓練集與驗證集
trainIdx = indices(1:trainCount);
valIdx = indices(trainCount+1:end);

trainImages = trainData(trainIdx);
trainMasks = groundTruthData(trainIdx);

valImages = trainData(valIdx);
valMasks = groundTruthData(valIdx);
```

接著由於匯入模型的資料需要特定的資料型態，所以我先將這些資料都匯出變成照片並存在對應的資料夾後，再透過 imageDataStore、pixelLabelDataStore 以及 pixelLabelImageDataStore 這些函數將資料以另一種方式存下來。

```
TrainImgsFolder = 'Train_Imgs_Data';
TrainMasksFolder = 'Train_Masks_Data';
ValImgsFolder = 'Val_Imgs_data';
ValMasksFolder = 'Val_Masks_Data';

if ~exist(TrainImgsFolder, 'dir')
    mkdir(TrainImgsFolder);  % 如果資料夾不存在，創建它
    for i=1:trainCount
        imwrite(trainImages{i}, fullfile(TrainImgsFolder, sprintf('Train_image_%d.tif', i)));
    end
end

if ~exist(TrainMasksFolder, 'dir')
    mkdir(TrainMasksFolder);  % 如果資料夾不存在，創建它
    for i=1:trainCount
        imwrite(trainMasks{i},fullfile(TrainMasksFolder, sprintf('Train_mask_%d.png', i)));
    end
end

if ~exist(ValImgsFolder, 'dir')
    mkdir(ValImgsFolder);  % 如果資料夾不存在，創建它
    for i=1:(all_data_num-trainCount)
        imwrite(valImages{i}, fullfile(ValImgsFolder, sprintf('Val_image_%d.tif', i)));
    end
end
if ~exist(ValMasksFolder, 'dir')
    mkdir(ValMasksFolder);  % 如果資料夾不存在，創建它
    for i=1:(all_data_num-trainCount)
        imwrite(valMasks{i},fullfile(ValMasksFolder, sprintf('Val_mask_%d.png', i)));
    end
end

% classNames = ["background", "blood_vessel", "glomerulus", "unsure", "black_edge"];
% pixelLabelID = [0 1 2 3 4];

classNames = ["background", "blood_vessel"];
pixelLabelID = [0 1];

train_imds = imageDatastore(TrainImgsFolder);
train_pxds = pixelLabelDatastore(TrainMasksFolder, classNames, pixelLabelID);
val_imds = imageDatastore(ValImgsFolder);
val_pxds = pixelLabelDatastore(ValMasksFolder, classNames, pixelLabelID);

dsTrain = pixelLabelImageDatastore(train_imds, train_pxds);
dsVal = pixelLabelImageDatastore(val_imds, val_pxds);
```

6. Setup U-Net Structure 建構 U-Net 神經網路

而在訓練模型的演算法，我選擇使用 U-Net。此 U-Net 架構是參考陽交大的盧家鋒老師在 MATLAB 進階程式語言的上課影片中的範例程式碼 [6]。

我參照了那位老師的架構建立了 3 層 Encoder 以及 3 層 Decoder 總共四層的 U 型神經網路架構，其中我將輸入層的 size 改成我們自己影像的尺寸 512*512*3，以及最後一層卷積層的 filter number 改成 2，因為我們模型最後只要分成兩類－background、blood_vessel。最後，輸出層要改成使用 dicePixelClassification Layer，這是因為我使用 pixelClassification Layer 訓練幾次模型之後，發現怎麼全部的測試資料預測結果都是 background，幾乎一點 blood_vessel 的像素都沒有，後來我才發現這是因為我們的背景與血管結構分配不均勻，光是背景就佔了絕大部分，這也代表我只要預測全部都是背景的話正確率就高達 80%，所以最後才調整成使用 dicePixelClassificationLayer 作為輸出層。

6. Setup U-Net Structure

Reference from Compulsory Course for the Undergraduate Students , Lecturer: Chia-Feng Lu (alvin4016@nycu.edu.tw)

```
imageSize = [512, 512, 3];  % 圖像尺寸

% 創建一個空的網路結構
lgraph = layerGraph();

% classWeights = [0.2, 1.0, 1.0, 1.0, 1.0]; % 降低背景權重，其他類別設為1

% 架構 U-Net 結構
tempLayers = [ % Encoder: 1 depth
    imageInputLayer(imageSize, "Name", "imageinput");
    convolution2dLayer([3 3], 2, "Name","conv_1", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_1");
    reluLayer("Name", "relu_1")

    convolution2dLayer([3 3], 16, "Name","conv_2", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_2");
    reluLayer("Name", "relu_2")

    convolution2dLayer([3 3], 16, "Name","conv_3", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_3");
    reluLayer("Name", "relu_3")];

lgraph = addLayers(lgraph, tempLayers);

tempLayers = [ % Encoder: 2 depth
    maxPooling2dLayer([5 5], "Name", "maxpool_1", "Padding", "same", "Stride", [2 2]);
    convolution2dLayer([3 3], 16, "Name","conv_4", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_4");
    reluLayer("Name", "relu_4")

    convolution2dLayer([3 3], 32, "Name","conv_5", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_5");
    reluLayer("Name", "relu_5")

    convolution2dLayer([3 3], 32, "Name","conv_6", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_6");
    reluLayer("Name", "relu_6")];

lgraph = addLayers(lgraph, tempLayers);

tempLayers = [ % Encoder: 3 depth
    maxPooling2dLayer([5 5], "Name", "maxpool_2", "Padding", "same", "Stride", [2 2])
    convolution2dLayer([3 3], 32, "Name","conv_7", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_7")
    reluLayer("Name", "relu_7")

    convolution2dLayer([3 3], 64, "Name","conv_8", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_8")
    reluLayer("Name", "relu_8")

    convolution2dLayer([3 3], 64, "Name","conv_9", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_9")
    reluLayer("Name", "relu_9")];

lgraph = addLayers(lgraph, tempLayers);
```

```
tempLayers = [ % Bridge: 4 depth
    maxPooling2dLayer([5 5], "Name", "maxpool_3", "Padding", "same", "Stride", [2 2])
    convolution2dLayer([3 3], 64, "Name","conv_10", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_10");
    reluLayer("Name", "relu_10")

    convolution2dLayer([3 3], 128, "Name","conv_11", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm11")
    reluLayer("Name", "relu_11")

    convolution2dLayer([3 3], 128, "Name","conv_12", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_12")
    reluLayer("Name", "relu_12")

    transposedConv2dLayer([3 3], 32, "Name", "transposed-conv_1", "Cropping","same","Stride",[2 2])];

lgraph = addLayers(lgraph, tempLayers);

tempLayers = [ % Decoder: 3 depth
    depthConcatenationLayer(2, "Name", "depthcat_3")
    convolution2dLayer([3 3], 64, "Name","conv_13", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_13")
    reluLayer("Name", "relu_13")

    convolution2dLayer([3 3], 64, "Name","conv_14", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm14")
    reluLayer("Name", "relu_14")

    convolution2dLayer([3 3], 64, "Name","conv_15", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_15")
    reluLayer("Name", "relu_15")

    transposedConv2dLayer([3 3], 32, "Name", "transposed-conv_2", "Cropping","same","Stride",[2 2])];

lgraph = addLayers(lgraph, tempLayers);

tempLayers = [ % Decoder: 2 depth
    depthConcatenationLayer(2, "Name", "depthcat_2")
    convolution2dLayer([3 3], 32, "Name","conv_16", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_16")
    reluLayer("Name", "relu_16")

    convolution2dLayer([3 3], 32, "Name","conv_17", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm17")
    reluLayer("Name", "relu_17")

    convolution2dLayer([3 3], 32, "Name","conv_18", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_18")
    reluLayer("Name", "relu_18")

    transposedConv2dLayer([3 3], 32, "Name", "transposed-conv_3", "Cropping","same","Stride",[2 2])];

lgraph = addLayers(lgraph, tempLayers);

tempLayers = [ % Decoder: 1 depth
    depthConcatenationLayer(2, "Name", "depthcat_1")
    convolution2dLayer([3 3], 32, "Name","conv_19", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_19")
    reluLayer("Name", "relu_19")

    convolution2dLayer([3 3], 16, "Name","conv_20", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm20")
    reluLayer("Name", "relu_20")

    convolution2dLayer([3 3], 16, "Name","conv_21", "Padding", "same")
    batchNormalizationLayer("Name", "batchnorm_21")
    reluLayer("Name", "relu_21")

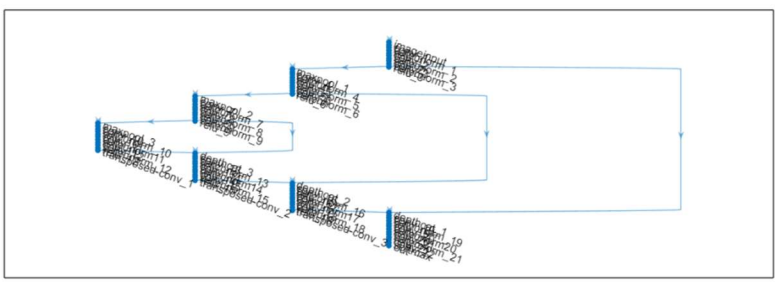
    convolution2dLayer([3 3], 2, "Name","conv_22", "Padding", "same")
    softmaxLayer("Name","softmax")
    dicePixelClassificationLayer('Name', 'out')]; % 使用 Dice Loss 適合不均衡分割
    % pixelClassificationLayer('Name','out','Classes',classNames,'ClassWeights',classWeights)]; % 因為過度於擬合背景，所以重新分配權重
    % pixelClassificationLayer("Name", "pixel-class")]; % 原本的

lgraph = addLayers(lgraph, tempLayers);

clear tempLayers

lgraph = connectLayers(lgraph, "relu_3", "maxpool_1");
lgraph = connectLayers(lgraph, "relu_3", "depthcat_1/in1");
lgraph = connectLayers(lgraph, "relu_6", "maxpool_2");
lgraph = connectLayers(lgraph, "relu_6", "depthcat_2/in1");
lgraph = connectLayers(lgraph, "relu_9", "maxpool_3");
lgraph = connectLayers(lgraph, "relu_9", "depthcat_3/in1");
lgraph = connectLayers(lgraph, "transposed-conv_1", "depthcat_3/in2");
lgraph = connectLayers(lgraph, "transposed-conv_2", "depthcat_2/in2");
lgraph = connectLayers(lgraph, "transposed-conv_3", "depthcat_1/in2");

clf; %  清除之前的圖
plot(lgraph);
```

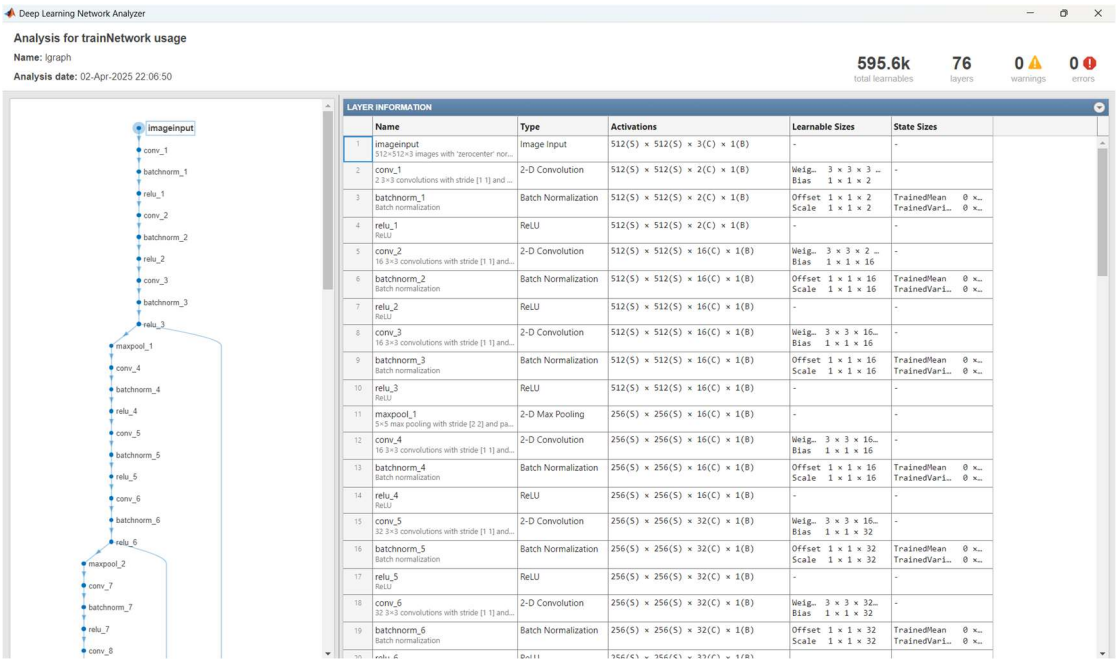


從上面 plot 此神經網路的架構圖中，我們也可以看出它的 U 型架構（轉個 90 度看）。以及當我們建構好神經網路後，建議再使用 analyzaNetwork 檢查此神經網路是否哪裡有錯誤，也方便我們清楚了解各個層次的架構，如下圖。

下面的 options 則是設定我們訓練時的超參數。其中，優化器我們選擇使用 sgdm（SGD with momentum），也就是將原本的隨機梯度下降法再加上上次的權重修正量，就像下降的時候會多一個動量，而使得我們可以輕鬆的越過局部極小值。再來，根據一般模型訓練的初始學習率都是從 0.001 開始，所以我也使用 0.001，比較穩固。而 MiniBatchSize 批次量則是根據我的 GPU 能承受多大的運算量所決定的數字，前幾次訓練的時候好幾次因為數字設太大而導致訓練到一半就因為記憶體不夠而強制終止訓練，但是設太小又會導致訓練要跑很久，所以最後是在經歷幾次訓練後才決定設定成 16。而 MaxEpochs 則是決定要訓練幾回合，基本上 20 回合才可以讓模型訓練的效果更好，但因為時間有限，所以我只設定成 10 回合。

```
analyzeNetwork(lgraph);

options = trainingOptions('sgdm', ...
    'MaxEpochs', 10, ...
    'MiniBatchSize', 16, ...
    'InitialLearnRate', 1e-3, ...
    'Shuffle', 'every-epoch', ...
    'LearnRateDropPeriod', 10, ...
    'DispatchInBackground', true, ...
    'ExecutionEnvironment', "GPU", ...
    'Verbose', true, ...
    'Plots', 'training-progress', ...
    'ValidationData', dsVal);  % 如果有驗證資料
```



7. Start Training with U-Net 開始訓練模型

接著當我們模型都架構好後，就可以根據我們準備好的資料去訓練模型了！訓練好的模型則會根據當下訓練的時間將模型自動存成對應的名字。

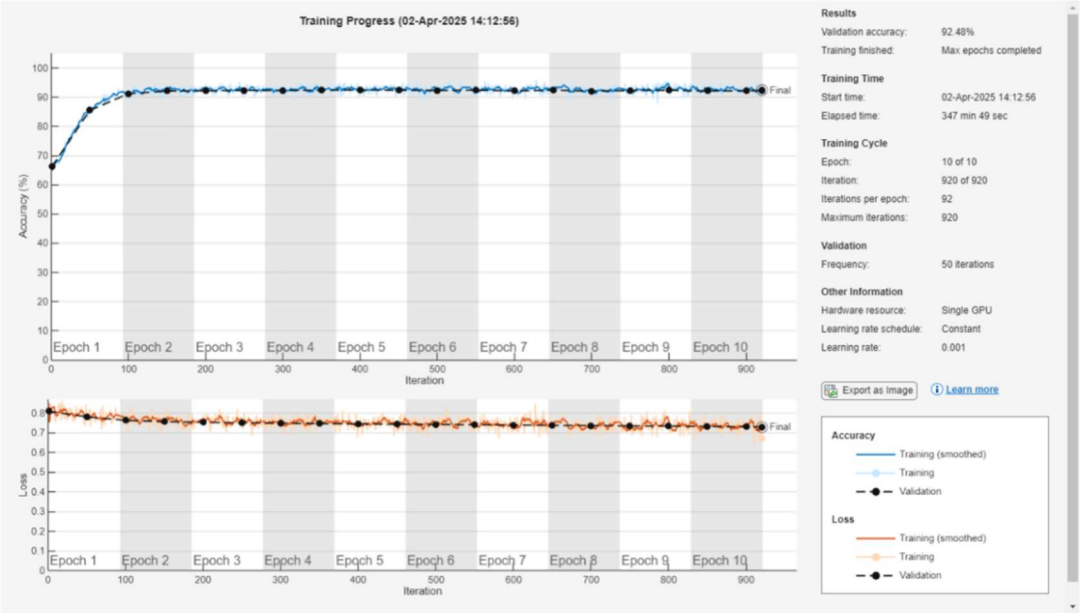
7. Start Training with U-Net

```
reset(gpuDevice);  % 重置 GPU

modelDateTime = string(datetime('now', 'Format','yyyy-MM-dd-HH-mm'));
[model, info] = trainNetwork(dsTrain, lgraph, options);
```

```
Starting parallel pool (parpool) using the 'Processes' profile ...
02-Apr-2025 14:11:48: Job Queued. Waiting for parallel pool job with ID 1 to start ...
Connected to parallel pool with 4 workers.
Initializing input data normalization.
|=========================================================================================================|
|  Epoch  |  Iteration  |  Time Elapsed  |  Mini-batch  |  Validation  |  Mini-batch  |  Validation  |  Base Learning  |
|         |             |   (hh:mm:ss)   |   Accuracy   |   Accuracy   |     Loss     |     Loss     |      Rate       |
|=========================================================================================================|
|       1 |           1 |       00:01:03 |      67.00% |      66.26% |       0.8526 |       0.8093 |          0.0010 |
|       1 |          50 |       00:18:42 |      86.03% |      85.52% |       0.7883 |       0.7809 |          0.0010 |
|       2 |         100 |       00:36:05 |      91.13% |      91.24% |       0.7500 |       0.7630 |          0.0010 |
|       2 |         150 |       00:53:21 |      92.47% |      92.16% |       0.7584 |       0.7572 |          0.0010 |
|       3 |         200 |       01:12:11 |      94.32% |      92.24% |       0.7878 |       0.7535 |          0.0010 |
|       3 |         250 |       01:30:17 |      93.02% |      92.36% |       0.7780 |       0.7507 |          0.0010 |
|       4 |         300 |       01:49:56 |      94.13% |      92.32% |       0.7802 |       0.7483 |          0.0010 |
|       4 |         350 |       02:07:38 |      93.63% |      92.42% |       0.7657 |       0.7465 |          0.0010 |
|       5 |         400 |       02:25:49 |      93.38% |      92.51% |       0.7400 |       0.7450 |          0.0010 |
|       5 |         450 |       02:43:03 |      92.98% |      92.41% |       0.7381 |       0.7438 |          0.0010 |
|       6 |         500 |       03:00:44 |      93.03% |      92.32% |       0.7649 |       0.7424 |          0.0010 |
|       6 |         550 |       03:19:31 |      93.95% |      92.43% |       0.7695 |       0.7407 |          0.0010 |
|       7 |         600 |       03:38:19 |      91.17% |      92.17% |       0.7132 |       0.7396 |          0.0010 |
|       8 |         650 |       03:55:56 |      92.51% |      92.43% |       0.7451 |       0.7376 |          0.0010 |
|       8 |         700 |       04:14:21 |      91.73% |      92.10% |       0.7520 |       0.7364 |          0.0010 |
|       9 |         750 |       04:32:16 |      92.09% |      92.33% |       0.7175 |       0.7349 |          0.0010 |
|       9 |         800 |       04:49:31 |      93.84% |      92.38% |       0.7261 |       0.7338 |          0.0010 |
|      10 |         850 |       05:07:46 |      93.46% |      92.18% |       0.7024 |       0.7317 |          0.0010 |
|      10 |         900 |       05:30:14 |      92.38% |      92.22% |       0.7431 |       0.7302 |          0.0010 |
|      10 |         920 |       05:47:08 |      91.83% |      92.28% |       0.6729 |       0.7299 |          0.0010 |
|=========================================================================================================|
Training finished: Max epochs completed.
```

從訓練過程的數據以及圖中，我們也可以觀察到基本上經歷過 100 個 Interation 後，模型的正確率以及誤差基本上就固定了。而此模型的正確率大約落在 92%。




```
save(strcat('trainedModel', modelDateTime, '.mat'), 'model'); % 將跑好的model存取下來
```

8. Predict the testing data 使用模型預測測試資料

接著就可以拿訓練好的模型去預測測試集資料了～

（如果要使用之前訓練好模型的資料也可以 load 進去使用。）

8. Predict the testing data

```
% load('trainedModel-XXXX-XX-XX-XX-XX.mat', 'net', 'options'); % 存取之前存下來的model，中間記得改成對應的時間
% load('trainedModel.mat', 'model');
Result = predict(model, test_imds);
```

9. Transfer Predict Results to Label Data 將預測結果轉換成標記資料

因為我們要將結果存下來，所以先創建資料夾以及存放資料的空間。

而由於預測的結果會以各個類別的機率形式存放在第 3 個維度下，所以我們要針對每一張測試集的照片，抓取第 3 維的資料後，比較每一個 pixel 在哪一個類別下擁有更高的機率，那那個 pixel 就會被預測成是那個類別，這樣我們就可以得到 mask 的資料。

接著就如同前面標註訓練集的資料一樣，我們先設定好類別的顏色以及透明度，再將 mask=1 也就是 blood_vessel 血管的部分弄成綠色，接著再將圖片存到資料夾去就好了

而 json 檔案的部分也是抓取 mask=1 的 x,y 座標讀入檔案，這樣一切就大功告成了！

9. Transfer Predict Results to Label Data

```
% 重新定義標籤類別與顏色 (RGB 格式，範圍 0~255)
class_colors = [0 255 0];    % 1 = blood_vessel (綠色)

% 設定遮罩的透明度
alpha = 0.7;

numImages = size(Result, 4); % 取得第四維度的資料：總影像數量

% 設定輸出資料夾
predictJsonsFolder = 'Predicted_json/';
if ~exist(predictJsonsFolder, 'dir')
    mkdir(predictJsonsFolder); % 建立資料夾（如果不存在）
end

predictImagesFolder = 'Predicted_images/';
if ~exist(predictImagesFolder, 'dir')
    mkdir(predictImagesFolder); % 建立資料夾（如果不存在）
end

testFolder = dir(fullfile('test','*.tif'));

% 初始化 Mask 和 結果 照片
predictedMasks = cell(1, numImages);  % 建立一個長度為 290 的 cell 陣列
predictedImage = cell(2, numImages); % (1,:)是直接覆蓋區域的圖；(2,:)的遮罩會呈現半透明

for i = 1:numImages
    predictedMasks{i} = zeros(512, 512, 'uint8');
    predictedImage{1,i} = test_imds.readimage(i);
    predictedImage{2,i} = test_imds.readimage(i);
end
```

```
for i = 1:numImages
    singleImageMask = Result(:, :, :, i); % 取出第 i 張影像的預測結果
    [~, predictedMasks{i}] = max(singleImageMask, [], 3); % 取機率最高的類別索引
    predictedMasks{i} = uint8(predictedMasks{i});  % 轉成 uint8

    mask = (predictedMasks{i} == 2);  % 找出標籤 1 的區域
    [y, x] = find(mask == 1);

    % 建立 JSON 結構
    jsonStruct = struct();
    jsonStruct.type = "blood_vessel";
    jsonStruct.coordinates = [x, y];  % 存成 (x, y) 格式
    jsonText = jsonencode(jsonStruct);% 轉換成 JSON 字串
    % 設定 JSON 檔名
    fileName = erase(testFolder(i).name,'.tif');
    jsonFileName = fullfile(predictJsonsFolder, [fileName, '.json']);

    % 儲存 JSON 檔案
    fid = fopen(jsonFileName, 'w');
    fprintf(fid, '%s', jsonText);
    fclose(fid);

    % 填充顏色 (class_colors(1,:) 是 1x3，代表 RGB 顏色)
    for c = 1:3
        color = double(class_colors(1, c));  % 轉換顏色為 double
        predictedImage{1,i}(:,:,c) = uint8(mask * color);  % 直接套用顏色
        predictedImage{2,i}(:,:,c) = uint8(double(predictedImage{2,i}(:,:,c)) + alpha * color * double(mask));
    end

    imwrite(predictedImage{1,i}, fullfile(predictImagesFolder, ['predicted_' fileName '_1.tif']));
    imwrite(predictedImage{2,i}, fullfile(predictImagesFolder, ['predicted_' fileName '_2.tif']));
end
```

3. 介紹演算法－U-Net：

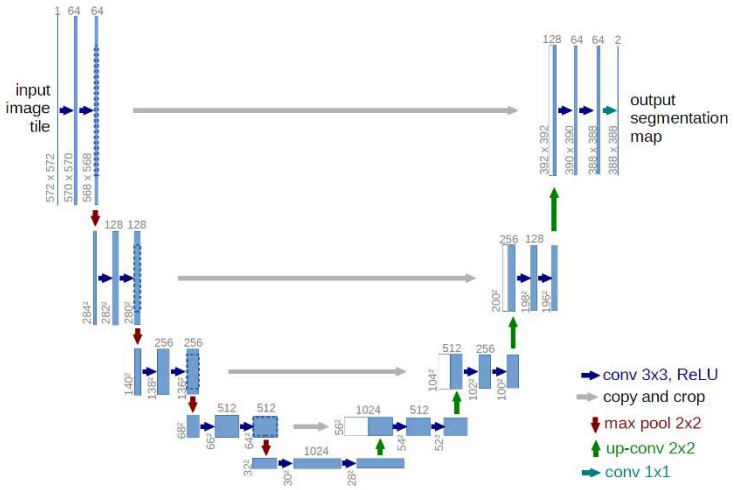
在訓練辨識微血管區域的模型中，我選擇使用 U-Net 作為我的演算法。

- **背景：**

  由 Olaf Ronneberger、Philipp Fischer 和 Thomas Brox 於 2015 年在德國弗萊堡大學開發，並使用此方法於當年的 ISBI 圖像分割競賽榮獲大賽冠軍。

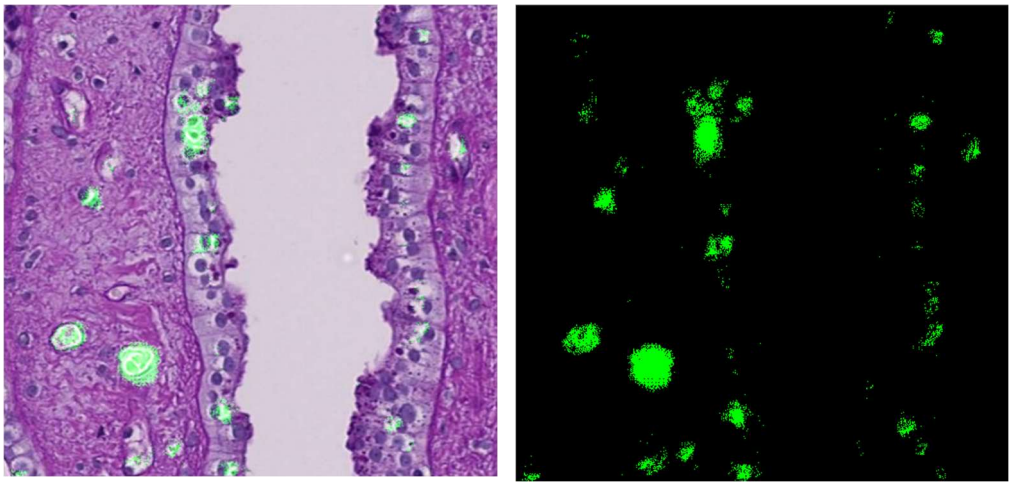
- **網路架構：**
  - U-Net 是基於全卷積神經網路（FCN）做出的語意分割演算法，其主要應用於生物醫學影像的分割處理。

    （FCN 和 CNN 的差異在於，CNN 最後的卷積層是全連接層，所以我們只能得到整張圖片的分類結果；但是 FCN 則將最後的全連接層換成卷積層，通過上採樣來保留空間資訊，從而對圖片中的每一個 pixel 做分類，而達到我們想要的分割效果。）
  - 此網路架構整體呈現 U 型，也因此得名為 U-Net，主要由 contraction path 收縮路徑和 expansive path 擴展路徑組成。
  - 架構左半側為收縮路徑，又稱編碼器（Encoder），是由無數個卷積、最大池化、以及 ReLU 函數所組成的 down sampling 操作，如同傳統的卷積網路，逐步減少特徵圖的空間維度，同時增加特徵的數量，從簡單的較大特徵圖逐漸萃取成複雜的較小特徵圖。
  - 架構右半側為擴展路徑，又稱解碼器（Decoder），是由無數個反卷積、卷積、以及 ReLU 函數所組成的 up sampling 操作，將前面經過萃取後的小特徵圖重建回去與原圖大小的新圖像，其中會與相對應層次的收縮路徑特徵圖資訊進行跳躍連接（skip connection），進而抓取到簡單特徵（也就是小區域）的定位，從而提高分割的精確度。

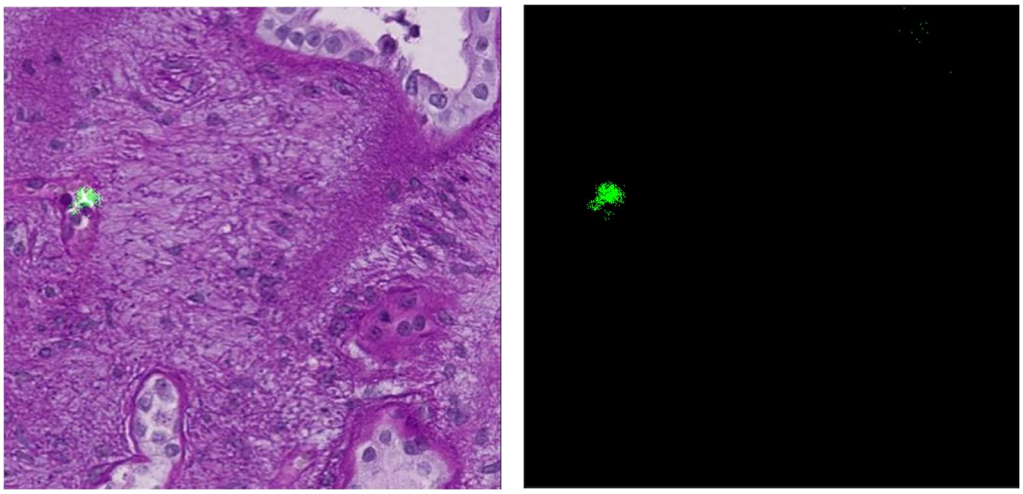


四、成果

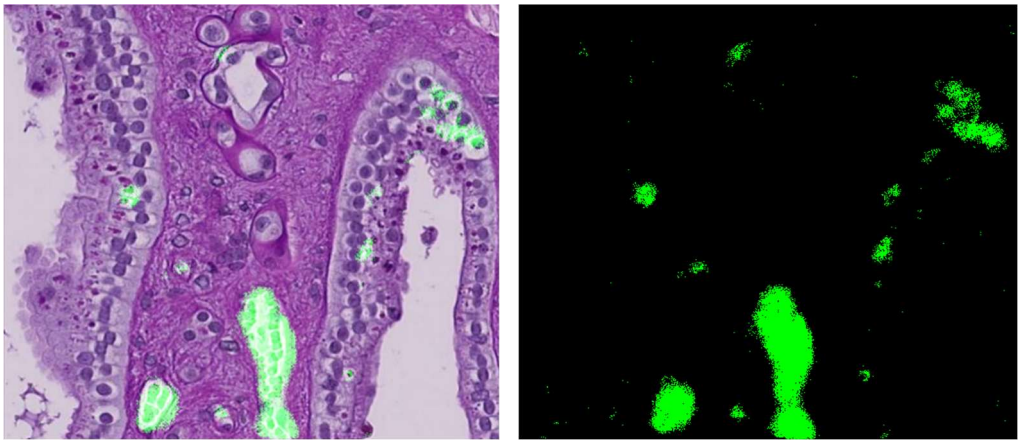
111111111111.tif：



777777777777.tif：



999999999999.tif：



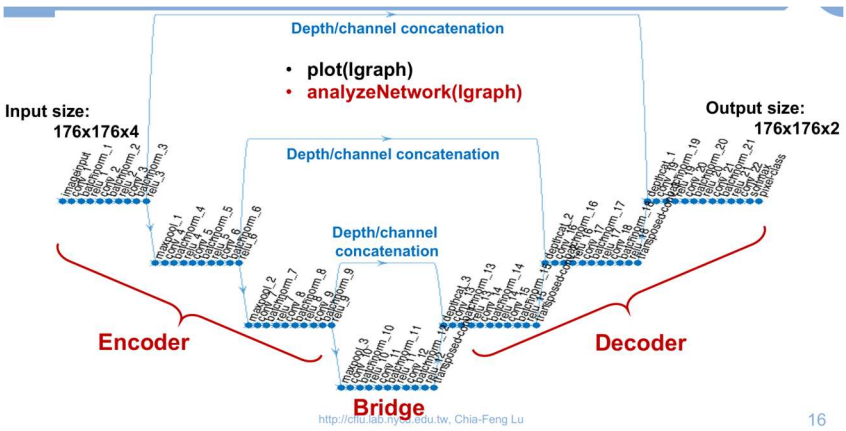
五、討論與感想

我認為在使用程式碼解決問題時，最核心的關鍵在於先建構並設計整體架構，也就是處理流程。有了清晰的流程後，才能更具體地規劃各個部分的實作內容。因此，在準備這次作業時，我首先設計了完整的架構，這也是我第一次親自操作並完成醫學影像分析。我參考了課堂上老師介紹的多種案例，以及其中的處理流程，再進一步調整，設計出適合本次目標的架構。

其中，一個與我原先預期不同的地方是標註資料的方式。我原本計畫使用 Medical Image Labeler 來標記影像，但由於我們的數據提供的是標記區域的外圍 (x, y) 座標，因此可以直接使用 poly2mask 來完成標註。既然 poly2mask 能夠有效處理這部分，我便選擇直接運用，而不再額外透過 Medical Image Labeler 進行標記。

此外，在使用 U-Net 演算法的過程中，我透過這次機會手刻了一個完整的 U-Net 架構，使我對其運作原理有了更深入的理解。尤其是各層之間的連接與操作，實作過程讓我更加體會其設計的精妙之處，這部分真的非常有趣！

以下是參考的架構示意圖 [6]：



最後，關於模型訓練的部分，確實有許多值得探討的細節。數據的前處理方式、超參數的調整等，每一項決策都可能對最終的模型表現產生顯著影響。如果時間允許，希望能進一步調整參數、觀察訓練結果，探索更多可能性。

最後，透過這次的作業讓我對於使用深度學習對醫學影像進行分析並建立模型更加的了解，除了理解到流程的各個步驟之外，中間的探討以及逐步調整細節的過程都是這次學習的精華，是一次收穫滿滿的作業！

六、參考資料

[1] 將病理影像資料統一色彩的方法：
https://juejin.cn/post/7163108775992229896

[2] Brief Introduction of Vahadane：
https://medium.com/@minhkhangle.phd/harmonizing-colors-across-institutions-an-introduction-to-vahadane-stain-normalization-in-digital-26dc97f8f31c

[3] Paper of Vahadane Stain Normalization：
https://ieeexplore.ieee.org/abstract/document/7460968

[4] 基於 GPU 加速的 Vahadane：
https://blog.csdn.net/CalvinTri/article/details/135429053
https://github.com/cwlkr/torchvahadane

[5] 劃分資料集
https://medium.com/nlp-tsupei/機器學習怎麼切分資料-訓練-驗證-測試集-f5a92576d1aa

[6] 建立 U-Net 架構：陽交大盧家鋒老師在 MATLAB 進階程式語言的上課影片
https://youtu.be/drF4L0jXFhY?si=IQS1DbD32HZwbfnE

[7] Paper of U-Net：U-Net: Convolutional Networks for Biomedical Image
https://arxiv.org/abs/1505.04597

[8] 介紹 U-Net 的網站
https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/
https://hackmd.io/@tarostudent99/BkNl8VVJu
https://tomohiroliu22.medium.com/深度學習 paper 系列-05-u-net-41be7533c934

[9] 其他參考資料
深度學習的 16 堂課 / CNN、RNN、GAN、DQN、DRL 看得懂、學得會、做得出！（ISBN：978-986-312-678-2）